

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

«Реализация REST API с помощью boilerplate»

Выполнил:

Федоров Даниил

Группа:

K3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- Использовать подходящий *boilerplate*
- С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализовать:
 - модели
 - представления
 - контроллеры
- Сделать полностью готовый API

Ход работы

Были реализованы модели базы данных в папке *src/models*. Модель *User* описывает общие данные пользователя: *email*, пароль, телефон и роль. Для разделения пользователей по ролям были созданы отдельные сущности *Seeker* и *Employer*. Сущность *Seeker* хранит данные профиля соискателя, а сущность *Employer* хранит данные профиля работодателя и компании.

Для резюме была создана сущность *Resume*. Она связана с профилем соискателя и хранит основную информацию о резюме: название, описание “о себе” и дату создания. Так как образование и опыт работы были вынесены в отдельные сущности, были дополнительно реализованы модели *Education* и *ResumeExperience*. Сущность *Education* хранит сведения об учебном заведении, степени, направлении обучения и датах обучения. Сущность *ResumeExperience* хранит информацию об опыте работы: компанию, должность, описание обязанностей и период работы.

Для вакансий была создана сущность *Vacancy*. Она связана с работодателем, отраслью и вариантом опыта работы. Вакансия хранит название, описание, требования, зарплату, город и дату создания. Для справочников были созданы сущности *ExperienceOption*, *Industry* и *Skill*. *ExperienceOption* используется для хранения вариантов требуемого опыта работы, *Industry* для отраслей, а *Skill* для навыков.

Так как навыки могут быть связаны и с резюме, и с вакансиями, были реализованы промежуточные сущности *ResumeSkill* и *VacancySkill*. Они позволяют организовать связь многие-ко-многим между резюме и навыками, а также между вакансиями и навыками. Такой подход позволяет

не дублировать навыки в базе данных и упрощает дальнейшее расширение проекта.

Для работы с откликами была создана сущность *ApplicationResponse*. Она связывает резюме соискателя и вакансию работодателя. В отклике хранится статус и дата создания. Также была реализована отдельная сущность *CoverLetter*, которая хранит сопроводительное письмо, связанное с конкретным откликом. Таким образом, сопроводительное письмо было вынесено в отдельную таблицу, что соответствует обновленной модели базы данных.

Для авторизации были реализованы регистрация, вход и выдача JWT-токена. В контроллере *src/controllers/auth.controller.ts* были созданы эндпоинты для регистрации соискателя, регистрации работодателя и входа в систему. При регистрации соискателя создается запись пользователя, профиль соискателя и пустое резюме. При регистрации работодателя создается запись пользователя и профиль работодателя. При входе проверяются *email* и пароль, после чего пользователю выдается JWT-токен с данными о его идентификаторе, роли и идентификаторе профиля.

Для формирования ответов API был создан файл *src/views/serializers.ts*. В нем описаны функции, которые преобразуют объекты TypeORM в JSON-ответы, соответствующие структуре API. Например, данные из формата *camelCase*, используемого в TypeScript, преобразуются в формат *snake_case*, используемый в Swagger и API. Также сериализаторы позволяют не отдавать лишние данные, например пароль пользователя.

Основные модели:

- *user.entity.ts* – общие данные пользователя.
- *seeker.entity.ts* – профиль соискателя.
- *employer.entity.ts* – профиль работодателя.
- *resume.entity.ts* – резюме соискателя.
- *education.entity.ts* – образование в резюме.
- *resume-experience.entity.ts* – опыт работы в резюме.
- *skill.entity.ts* – справочник навыков.
- *resume-skill.entity.ts* – связь резюме и навыков.
- *experience-option.entity.ts* – справочник вариантов опыта работы.
- *industry.entity.ts* – справочник отраслей.

- vacancy.entity.ts – вакансия.
- vacancy-skill.entity.ts – связь вакансии и навыков.
- response.entity.ts – отклик на вакансию.
- cover-letter.entity.ts – сопроводительное письмо.
- enums.ts – перечисления ролей пользователей и статусов откликов.

Основные контроллеры:

- auth.controller.ts – регистрация соискателя, регистрация работодателя и вход в систему.
- seeker-profile.controller.ts – получение и обновление профиля соискателя, получение и обновление резюме.
- employer-profile.controller.ts – получение и обновление профиля работодателя.
- reference-data.controller.ts – получение справочников опыта работы, отраслей и навыков.
- vacancies.controller.ts – работа с вакансиями: список, детальная информация, создание, обновление, удаление.
- responses.controller.ts – создание откликов, просмотр списка откликов работодателем и детальный просмотр отклика.

Описание основных эндпоинтов

Авторизация

- POST /api/v1/auth/register/seeker – регистрация соискателя. Создает пользователя, профиль соискателя и пустое резюме.
- POST /api/v1/auth/register/employer – регистрация работодателя. Создает пользователя и профиль работодателя.
- POST /api/v1/auth/login – вход в систему. Проверяет email и пароль, после чего возвращает JWT-токен.

Профиль соискателя

- GET /api/v1/seekers/:userId/profile – получение профиля соискателя.
- PATCH /api/v1/seekers/:userId/profile – обновление профиля соискателя.
- GET /api/v1/seekers/:userId/resume – получение резюме соискателя.
- PATCH /api/v1/seekers/:userId/resume – обновление резюме, образования, опыта и навыков.

Профиль работодателя

GET /api/v1/employers/:userId/profile – получение профиля работодателя.

PATCH /api/v1/employers/:userId/profile – обновление профиля работодателя.

Справочники

GET /api/v1/reference/experiences – получение вариантов опыта работы.

GET /api/v1/reference/industries – получение списка отраслей.

GET /api/v1/reference/skills – получение списка навыков.

Вакансии

GET /api/v1/vacancies – получение списка вакансий с поиском, фильтрацией, сортировкой и пагинацией.

GET /api/v1/vacancies/:vacancyId – получение детальной информации о вакансии.

GET /api/v1/employers/:employerId/vacancies – получение списка вакансий конкретного работодателя.

POST /api/v1/employers/:employerId/vacancies – создание вакансии работодателем.

PATCH /api/v1/employers/:employerId/vacancies/:vacancyId – обновление вакансии работодателем.

DELETE /api/v1/employers/:employerId/vacancies/:vacancyId – удаление вакансии работодателем.

Отклики

POST /api/v1/vacancies/:vacancyId/responses – создание отклика на вакансию соискателем.

GET /api/v1/employers/:employerId/responses – получение списка откликов на вакансии работодателя.

GET /api/v1/responses/:responseId – получение детальной информации об отклике.

Вывод

В результате выполнения лабораторной работы было реализовано REST API для сайта поиска работы с использованием Express, TypeScript и TypeORM. Была создана структура проекта, настроено подключение к PostgreSQL и реализованы основные сущности базы данных в соответствии с предметной областью. Для удобства работы с API были реализованы справочники опыта работы, отраслей и навыков. Для списков вакансий и откликов были добавлены пагинация и сортировка. Ответы API были приведены к единому формату с помощью функций сериализации.